Assignment 2: BBS Webserver

Software Engineering - UATX - Spring 2026

Due	Wednesday, April 22, 2026, 11:59pm
Estimated time	10-15 hours (with coding agent)
Prereqs	Assignment 1 (BBS) and lectures 3.1 (REST & HTTP) and 3.2 (FastAPI)
Submit	Branch + PR on the class repo

Background

Your BBS from Assignment 1 works, but it only works on your laptop. To let anyone on the internet read and post, you need a web server. In this assignment, you will wrap your BBS in a REST API using FastAPI. The database layer from A1 stays the same. You are adding a new layer on top.

The CLI version does not go away. You end up with two ways to talk to the same database: the commands from A1, and a set of HTTP endpoints you can hit from curl, a browser, or (in Assignment 3) a React frontend.

This assignment ships with a partial conformance script called `verify_api.py`. It tests most of the endpoints automatically. You will extend it to cover the rest before submitting. The extension is part of the grade.

Setup

Copy the starter file from `starter/assignment2/verify_api.py` in the class repo into your working directory alongside a README stub. Your final directory should look like this:

```
assignments/bbs-webserver/<your-github-username>/
  main.py           # your FastAPI application
  db.py             # engine + init_db()
  verify_api.py     # provided - you will extend it
```

```
  requirements.txt  # one dependency per line (see below)
  README.md
```

You can reuse your `db.py` from A1 with minor changes (see the schema change section below). Your `main.py` is new.

We have not formally covered `requirements.txt` in lecture yet - we will in week 7 when we start deploying things. For now, it is a plain text file with one package name per line, which `pip install -r requirements.txt` reads to install everything at once. For this assignment, yours should contain:

```
fastapi
uvicorn
httpx
```

`httpx` is the HTTP client used by `verify_api.py`. Pydantic comes with FastAPI and does not need its own line.

The API

Resources

The API is organized around two resources: `/users` and `/posts`. Resist the urge to map your A1 CLI commands to endpoints one-for-one. Think in resources. A user is a thing you can create, read, and list. A post is a thing you can create, read, list, search, and delete. The endpoints below follow from that framing.

Endpoints

Method	Path	Purpose
POST	/users	Create a user. Body: `{"username": "..."}`
GET	/users	List all users
GET	/users/{username}	Get one user by username
GET	/users/{username}/posts	Get all posts by that user
POST	/posts	Create a post. Body: `{"message": "..."}`.

		Requires `X-Username` header
GET	/posts	List posts. Supports `?q=`, `?limit=`, `?offset=`
GET	/posts/{id}	Get one post by id
DELETE	/posts/{id}	Delete a post (hard delete)

Response shapes

Field names are exact and checked by the verifier. Bronze responses look like this:

```
# Post
{"id": 1, "username": "alice", "message": "hi", "created_at": "2026-04-13T14:01:32"}

# User
{"username": "alice", "created_at": "2026-04-13T14:01:32"}
```

List endpoints return bare JSON arrays. No envelope objects in bronze.

Validation

Use Pydantic models for request body validation and FastAPI's `Query()` for query parameter validation.

- **message** - non-empty string, max 500 characters. Bad input returns 422.
- **username** - 3 to 20 characters, regex `^[a-zA-Z0-9_]+$`. Bad input returns 422.
- **limit** - integer, 1 to 200, default 50. Out-of-range returns 422 (via `Query(ge=1, le=200)`).
- **offset** - integer, >= 0, default 0. Negative value returns 422 (via `Query(ge=0)`).

Status codes

Situation	Code
Successful create (POST /users, POST /posts)	201
Successful read (any GET)	200

Successful delete	204
Missing `X-Username` header on POST /posts	400
Resource not found (post id, username)	404
POST /posts with unknown username	404
POST /users with duplicate username	409
Pydantic validation failure on body or query params	422

Note: out-of-range `limit` and `offset` return 422, not 400. That is what Pydantic's `Query(ge=..., le=...)` produces. The 400 on missing `X-Username` is custom code in your handler.

Error body

Use the FastAPI default error shape: `{"detail": "..."}`. Same as what lecture 3.2 uses. You do not need to invent a custom error format.

The X-Username header

When a client creates a post, the server needs to know who is posting. In A2 we use a custom request header: `X-Username: alice`. Every `POST /posts` request must include this header. If it is missing, return 400. If the username does not exist in the database, return 404.

This is not authentication. Anyone can put any username in that header and the server will accept it. We are not checking passwords or tokens. The pattern exists to introduce the idea that HTTP requests carry identity metadata, even before we have real auth. We will get to real auth in a later lecture.

Schema change from A1

A1 auto-created users on first post. If you posted as `alice` and there was no user named `alice`, A1 would create her silently. A2 does not do this. `POST /posts` with `X-Username: alice` when `alice` does not exist returns 404. Users must be created via `POST /users` first.

This is a real behavior change, not just a wrapper. If you port your A1 database logic verbatim, your API will produce wrong results. Review your `db.py` post-creation path and remove the auto-create logic before building the API layer on top of it.

The verifier

How to run

You need two terminals. In the first:

```
uvicorn main:app --port 8000
```

In the second:

```
python verify_api.py
```

The script prints a `PASS` or `FAIL` line for each check, then exits non-zero if anything failed. It uses random username suffixes on every run, so you do **not** need to delete `bbs.db` between runs. If you want a truly clean slate, stop the server, delete `bbs.db`, and restart.

What the provided script covers

The shipped `verify_api.py` verifies the following:

User checks:

- POST /users returns 201 on success, with `username` and `created_at` in the response body
- POST /users returns 409 on duplicate username
- POST /users returns 422 for a too-short username, invalid characters, or missing username field
- GET /users returns 200 and includes the created users
- GET /users/{username} returns 200 for an existing user, 404 for a missing one

Post checks:

- POST /posts with valid `X-Username` header returns 201, with `id`, `username`, `message`, `created_at` in the response
- POST /posts without `X-Username` header returns 400
- POST /posts with a username that does not exist returns 404
- POST /posts with empty message, oversized message (501 chars), or missing message field returns 422
- GET /posts returns 200 and a JSON array
- GET /posts/{id} returns 200 for an existing post, 404 for a missing one
- GET /users/{username}/posts returns 200 with only that user's posts, 404 for missing user

Search checks:

- GET /posts?q=keyword returns 200 and only matching posts

What you add for bronze

> **Required for bronze.** Three function stubs in `verify_api.py` are labeled `STUDENT TODO #1`, `#2`, and `#3`. You must implement all three and uncomment the calls in `main()`.

TODO #1 - `run_delete_checks(c, state)` should verify:

- DELETE on an existing post returns 204
- After DELETE, GET on the same id returns 404
- DELETE on a post id that does not exist returns 404

`state["alice_post_id"]` holds a post id created earlier in the run. You can use it (or create a new post specifically to delete). For the 404 case, pick a post id that is very unlikely to exist (e.g., 99999999).

TODO #2 - `run_pagination_checks(c, state)` should verify:

- GET /posts?limit=N returns at most N items
- GET /posts?offset=K skips the first K items
- GET /posts?limit=0 returns 422
- GET /posts?limit=500 returns 422
- GET /posts?offset=-1 returns 422

TODO #3 - `run_field_shape_checks(c, state)` should verify that your response bodies contain *exactly* the fields the spec lists, with no extras and nothing missing. Specifically:

- A user object (from `POST /users`, `GET /users/{username}`, and items in `GET /users`) has exactly `{username, created_at}`
- A post object (from `POST /posts`, `GET /posts/{id}`, and items in `GET /posts`) has exactly `{id, username, message, created_at}`
- No stray fields. An extra `email`, `updated_at`, or `user_id` is a fail.

This check exists because it is very easy to ship an endpoint that "works" in the happy path but leaks extra fields from your database rows. Reading the spec and comparing it to what your API actually returns is a real engineering skill - do it yourself, do not just ask your agent to make the check pass.

You may add more assertions if you catch edge cases the spec does not mention. Mention them in your README.

A reminder about PATCH (silver only)

> **PATCH** is the partial-update sibling of PUT: you send only the fields you want to change, and fields you omit are left alone. Silver uses PATCH for editing user bios and post messages. If you are targeting bronze, you can skip this section.

Tiers

> **Bronze**
> All 8 endpoints work per the spec above. The shipped `verify_api.py` passes. You implemented `run_delete_checks()`, `run_pagination_checks()`, and `run_field_shape_checks()`, uncommented all three calls, and every assertion passes. README covers all required sections.

> **Silver**
> Everything in bronze, plus:
>
> - **User profile fields.** Add `bio` (optional string, max 200 chars) and `post_count` (computed from the posts table) to all user responses: `{"username": "alice", "created_at": "...", "bio": "wolf in dog suit", "post_count": 14}`

- **PATCH /users/{username}** to update bio. Returns 200 on success, 404 if user not found, 422 if the bio exceeds the limit.
- **PATCH /posts/{id}** to edit message text. Adds `updated_at` to the post response. You must pick an ownership policy: enforce via `X-Username` header match (only the original author can edit), or allow anyone to edit (defensible since `X-Username` is not real auth anyway). Document your choice in the README.
- **GET /posts?username=alice** to filter posts by author. Composable with `?q=` and pagination.
- Extend `verify_api.py` with assertions for the silver features you added.

Gold

Everything in silver, plus one of the following (or something of comparable scope):

- **Cursor-based pagination.** Replace `?offset=` with `?cursor=` on GET /posts. The cursor encodes the last-seen post id. Response changes to an envelope: `{"posts": [...], "next_cursor": "eyJpZCI6IDUwfQ=="}`. README explains why cursors handle concurrent inserts better than offsets.
- **Boards/topics.** Add `/boards` and `/boards/{name}/posts` endpoints. Posts gain a `board` field. Requires a schema migration and a second resource.
- **/feed endpoint.** GET /feed returns the N most recent posts across all users. Optional `?since=<timestamp>` to get only newer posts.
- **Reactions.** `POST /posts/{id}/reactions` with `{"username": "...", "kind": "+1"}` and `DELETE /posts/{id}/reactions/{username}`. Introduces an association table.
- **Surprise us.** Build something you would want to show off.

Extend `verify_api.py` with assertions for your gold feature too.

README requirements

1. **How to run** - list dependencies, the uvicorn command, any setup steps (e.g., if you need to run a migration from your A1 database).
2. **Tier targeted** - bronze, silver, or gold.
3. **Design decisions** - at least 3 bullets. Good topics: why you chose this URL shape for a given endpoint, why a particular status code, hard vs soft delete and why, raw SQL vs ORM (did you keep A1's raw SQL or switch to the ORM?).

4. **Schema changes from A1** - show the `CREATE TABLE` diff inline. Even if the change is small (e.g., removing the auto-create-user behavior from your insert logic), say what changed and why.
5. **What you added to verify_api.py** - list your new assertions and any edge cases you checked beyond what the spec required.
6. **A short paragraph on X-Username and auth.** The header is not authentication - anyone could send any username. What would need to change for real authentication? You do not need the right answer. The goal is to think about the question before later lectures answer it.
7. **Silver/gold only:** what you added and why.

What to submit

1. Create a branch: `git checkout -b bbs-webserver-<your-github-username>`
2. Put your code in a directory: `assignments/bbs-webserver/<your-github-username>/`
3. Push and open a PR titled "BBS Webserver - <your-name>"

A note on coding agents

You are encouraged to use your coding agent throughout this assignment. The verifier is the floor - your agent can iterate against it until all checks pass. But the verifier cannot check your design decisions, your README, or what you chose to add to the verification script itself. Those are the ceiling. The ownership policy for PATCH, the reflection on X-Username as a non-authentication pattern, the edge cases you decided were worth asserting - those are yours to think through. The README sections are where you demonstrate you understood what you built, not just that an agent built something that passes the script.

Grading

Bronze	Satisfactory: you demonstrated the core skills
Silver	Good: you extended the design thoughtfully
Gold	Excellent: you built something impressive